G GRIT · Interview Hub

Technical & product manual.

A complete reference for the GRIT AI Product Engineer (Round 1) clearance prototype: the product decisions, domain rules, data model, concurrency design, security posture, test coverage and operating runbook.

Next.js 16 · React 19 | PostgreSQL · Prisma 7 | Server actions · Zod | 44 automated tests | Three roles | Production build verified

DOCUMENT
Technical & product overview

VERSION
1.0 · 11 November 2026

PREPARED FOR
Assignment interview walkthrough

REPOSITORY
`grit-interview-hub`

GRIT — Interview Hub

Round 1 · AI Product Engineer

NAVIGATION

00 What is in this document

01 Product summary
02 Users and demo personas
03 End-to-end flows
04 Screens and routes
05 Domain rules and state machines
06 Booking under contention
07 Availability and slot generation
08 Evaluation and publication
09 Badges, retakes and integrity
10 Notifications and exact copy
11 Audit and history
12 Data model reference
13 Server actions catalogue
14 Analytics and metrics
15 Security and access control
16 Time handling
17 Testing
18 Interface and accessibility
19 Operating runbook
20 Limitations and next steps
21 Interview questions and answers
A Policy constants
B Source map

DOMAIN SERVICES	STATE MACHINES	BOOKING DEFENCES
12	7	3
Rules isolated from React	Every write is checked	Locks, indexes, idempotency

> **STATUS STATEMENT**
> The application is complete and verified locally. A public deployment is prepared but not published because the hosting account and hosted database require credentials that are not available in this environment. No live URL is claimed in this document.

SECTION 01

01 Product summary

The problem. GRIT runs a practical clearance for AI Product Engineer candidates. It looks like a simple sequence — finish skills, book an interview, receive a result — but it is a coordination problem with contractual consequences. Slots are scarce and contended, results unlock a badge, retakes have waiting periods, integrity cases can revoke something already awarded, and every decision must remain explainable later.

The product. Interview Hub is a role-based workspace for the three people in that loop. It replaces spreadsheets and email threads with a system of record that enforces the rules at the domain and database layers, then explains each state in plain language.

PRODUCT PRINCIPLES

- **One next step.** The student dashboard derives a single highest-priority action instead of exposing every control.
- **No accidental commitments.** Booking is confirmed in a dialog that restates slot, interviewer and cancellation policy. Nothing is held while the dialog is open.

WHAT IS DELIBERATELY IN SCOPE

- Eligibility derived from required skills
- Atomic slot booking with a visible confirmation
- Interviewer availability, daily schedule, structured evaluation
- Operations publication, badges, retakes, malpractice, audit
- Analytics computed from the database

— **Honest error states.** Losing a race produces a specific message, not a generic failure.
— **History over overwrite.** Cancellations, reschedules, results and badge changes remain on record.
— **Queues, not tables.** Operations work through the things that need a decision.

— Before/after result comparison built on real records

WHAT IS DELIBERATELY OUT OF SCOPE

— Production identity provider and passwords
— Email or SMS delivery
— Multi-timezone slot creation
— File uploads for evidence

The student journey in six stages

#	STAGE	DERIVED FROM	WHAT THE STUDENT SEES
1	Skills	Required skills marked `COMPLETED`	Progress count and any missing milestone
2	Eligibility	`StudentProfile.eligibilityStatus`	Unlocked, pending skills, or revoked reason
3	Booking	Confirmed booking (or none)	Slot picker or current booking details
4	Interview	Interview status and scheduled time	Preparation guidance and interview details
5	Result	Evaluation / published result	Awaiting publication or published outcome
6	Badge	`BadgeAward.status`	Active, suspended, nullified, or not awarded

The journey is rendered by a shared builder used by both the live student dashboard and the before/after comparison screen, so the two cannot drift.

Before and after a result

The assignment required the student home and affected screens to be shown before and after a result inside the prototype. `/result-impact` renders three real seeded students side by side: Kabir Shah (interview complete, awaiting publication), Meera Krishnan (passed, badge awarded) and Rohan Verma (failed, dated retake window). Each panel shows the same six-stage tracker and the exact next-action card the dashboard would produce for that state.

> **WHY THIS SCREEN EXISTS**
> Result states are the emotional core of the product. Showing them against live rows — not mocked screenshots — proves that the empty, pending, passed and failed states are all designed, reachable and consistent with the domain.

SECTION 02

02 Users and demo personas

ROLE	JOB TO BE DONE	PRIMARY SURFACES
STUDENT	Understand the next step, book without mistakes, read outcomes, act on retakes	Overview, booking, interviews, badge, notifications
INTERVIEWER	Run the day, keep availability accurate, record a structured outcome	Day view, schedule, availability, history, evaluation
ADMIN	Keep the pipeline honest: publish, reassign, review, investigate	Command centre, students, bookings, interviewers, results, retakes, malpractice, analytics, audit

Demo access

Authentication is a signed session cookie issued by a persona picker. There are no passwords and no email delivery. The login screen offers **Continue as Student**, **Continue as Interviewer** and **Continue as Interview operations**, plus a searchable list of every seeded scenario. Setting `DEMO_MODE=false` disables both the picker and the reset action; `/result-impact` then returns 404.

Seeded scenarios (23 identities)

PERSON	ROLE	SEED STATE SHOWN IN THE PICKER
Aarav Mehta	Student	Eligible, not booked — the default student start
Ishita Rao	Student	Interview tomorrow · reassignment needed
Kabir Shah	Student	Result awaiting publication
Meera Krishnan	Student	Passed · badge awarded
Rohan Verma	Student	Failed · retake request pending
Ananya Iyer	Student	Cancelled booking
Dev Malhotra	Student	Not eligible · 2 of 5 skills
Sara Khan	Student	Badge awarded · case under review
Nikhil Joshi	Student	No-show recorded
Priya Nair	Student	Rescheduled interview
Arjun Desai	Student	Badge nullified · confirmed case
Tanya Bose	Student	Eligible for 9 days · unbooked
Aisha Siddiqui	Student	Eligibility revoked
Varun Pillai	Student	Retake approved · ready to book
Riya Sen	Student	Interview tomorrow · full day
Manav Gupta	Student	Interview tomorrow · full day
Sneha Pillai	Student	Interview tomorrow · full day
Neel Bhatt	Student	Retake later · waiting period
Vikram Sethi	Interviewer	Open slots and evaluation history
Neha Kapoor	Interviewer	Reviewing a blocked availability day
Aditya Menon	Interviewer	Fully booked tomorrow
Fatima Sheikh	Interviewer	Blocked window with bookings
Priyanka Raman	Operations	Full operations access

Seeded data is rebuilt by the reset action, which truncates all 18 application tables and reseeds the scenarios in one transaction.

SECTION 03

03 End-to-end flows

Student: from skills to badge

1. Skills and eligibility are shown as derived state. If ineligible, the dashboard names the missing milestones; if revoked, it shows the operations reason.

2. The student opens **Book interview**. The screen lists real open slots grouped by day, with interviewer, time and timezone. A preview mode (`?preview=1`) lets a blocked student inspect availability without implying they can book.
3. Selecting **Book** opens a confirmation dialog: slot, interviewer, and the four-hour cancellation policy. Nothing is reserved while it is open. The request carries a generated idempotency key.
4. On success, the dashboard and booking screen become guidance: date, time, preparation advice, and a link to interview details. A booking-confirmed notification is stored.
5. After the interviewer submits an evaluation, the student sees “awaiting publication”. After operations publishes, the result, feedback and badge state appear on the interviews and badge screens.
6. If the outcome is not a pass, the dashboard shows the retake window and offers a retake request with a written reason.

Interviewer: run the day

1. The day view shows today’s interviews, upcoming sessions, open/booked slot counts and interviews that need an evaluation.
2. Availability is managed per date with a window, slot length (15–180 minutes) and an optional break. Creating or editing generates real slots; overlapping windows and past dates are rejected.
3. Opening an assigned interview shows candidate context and a structured form: three 0–100 scores, strengths, improvements, internal notes and a recommendation (PASS / FAIL / RETAKE_LATER). A scheduled-in-the-future banner warns against premature submission.
4. Submission is terminal: the interview moves to result pending, the booking completes, and operations receives the publication item.

Operations: work the queues

1. **Results:** publish an evaluated interview with a feedback summary. Publication is terminal and awards the badge on a pass.
2. **Bookings:** reassign after an availability block, cancel with a reason, or record a no-show.
3. **Retakes:** approve or reject a student request with a decision note. Approval reopens booking before the waiting period ends.
4. **Integrity:** report a case, move it through review, confirm or reject it, and suspend or nullify a badge when confirmed.
5. **Students:** revoke or restore eligibility with an audited reason.
6. **Analytics and audit:** read funnel, outcomes, workload and the append-only decision trail.

> **DESIGNED FAILURE PATHS**
> Every flow has a visible failure state: lost slot race, booking with revoked eligibility, cancellation inside four hours, evaluating the same interview twice, publishing without an evaluation, double publication, booking during a retake wait, and a suspended badge. These are treated as first-class product states, not exceptions.

SECTION 04

04 Screens and routes

ROUTE	ACCESS	PURPOSE AND KEY ELEMENTS
`/`	Public	Landing page: product story, role cards, link to the before/after comparison
`/login`	Public (demo)	Role tabs, persona picker with scenario search, link to result comparison
`/result-impact`	Public (demo only)	Three seeded result states with shared journey and next-action builders

ROUTE	ACCESS	PURPOSE AND KEY ELEMENTS
/dashboard	Authenticated	Redirects to the role home
/student	Student	Next step, clearance path, eligibility, pending evaluation, skills, notifications
/student/book	Student	Availability by day, preview mode, confirmation dialog, existing-booking guidance
/student/interviews	Student	Upcoming and past interviews, results, feedback, cancel/reschedule/retake actions
/student/badge	Student	Badge identity, status, award date and verification context
/interviewer	Interviewer	Today, upcoming, needs evaluation, slot utilisation
/interviewer/interviews	Interviewer	Schedule with outcomes, awaiting publication
/interviewer/interviews/[id]	Assigned interviewer	Candidate context, start session, structured evaluation
/interviewer/availability	Interviewer	Create/edit/block/delete windows; slot count per window
/interviewer/history	Interviewer	Completed evaluations and scores
/notifications	Authenticated	Full notification history, mark one/all read
/admin	Operations	Command centre: KPIs, eligibility-unbooked, reassignments, retakes, integrity, audit
/admin/students	Operations	Student roster, eligibility actions, profile drill-down
/admin/students/[id]	Operations	Skills, bookings, interviews, results, badge and audit for one student
/admin/bookings	Operations	Confirmed bookings, reassignment, cancel, no-show
/admin/interviewers	Operations	Availability, workload and active state
/admin/results	Operations	Awaiting publication and published results
/admin/retakes	Operations	Retake requests and decisions
/admin/malpractice	Operations	Report, review, confirm/reject, badge action
/admin/analytics	Operations	Bookings trend, outcomes, interviewer workload, funnel
/admin/audit	Operations	Append-only trail with actor, action and before/after values

There is no middleware file. Protection is applied in each page and each server action, so a new route cannot rely on a global rewrite for safety. The authenticated layout adds the role shell; the guards decide access.

SECTION 05

05 Domain rules and state machines

All transitions live in `src/lib/state-machine.ts`. Writers call `assertTransition`; an illegal move throws a `DomainError` with code `STATE` and a human sentence. This concentrates the rules in one testable place and prevents a page from inventing its own workflow.

Interview

FROM	ALLOWED TO	TRIGGERED BY
SCHEDULED	IN_PROGRESS, COMPLETED, RESULT_PENDING	Interviewer start / evaluation
IN_PROGRESS	COMPLETED, RESULT_PENDING	Evaluation submission
COMPLETED	RESULT_PENDING, RESULT_PUBLISHED	Evaluation / publication
RESULT_PENDING	RESULT_PUBLISHED	Operations publication
RESULT_PUBLISHED	— terminal —	No further moves

Booking and slot

BOOKING	ALLOWED TO
CONFIRMED	CANCELLED, RESCHEDULED, COMPLETED, NO_SHOW
CANCELLED	terminal
RESCHEDULED	terminal
NO_SHOW	terminal
COMPLETED	terminal

SLOT	MEANING
OPEN	Bookable
BOOKED	Held by a confirmed booking
CANCELLED	Withdrawn or its time passed
BLOCKED	Reserved status for blocked windows (open slots are set CANCELLED on block)

Badge

FROM	ALLOWED TO	NOTES
NOT_EARNED	PENDING_CLEARANCE	Model transition table support
PENDING_CLEARANCE	AWARDED, NULLIFIED	Awards are created by publication
AWARDED	SUSPENDED, NULLIFIED	Requires a written note
SUSPENDED	AWARDED, NULLIFIED	Restore or escalate
NULLIFIED	terminal	Result remains on record

Retake, malpractice and eligibility

MACHINE	TRANSITIONS
Retake	PENDING → APPROVED | REJECTED; APPROVED → SCHEDULED → COMPLETED; REJECTED terminal
Malpractice	REPORTED → UNDER_REVIEW | REJECTED; UNDER_REVIEW → CONFIRMED | REJECTED; CONFIRMED/REJECTED → CLOSED
Eligibility	NOT_ELIGIBLE → ELIGIBLE; ELIGIBLE → ELIGIBILITY_REVOKED; ELIGIBILITY_REVOKED → ELIGIBLE

Booking eligibility, in order

#	CHECK	RULE AND USER-FACING CONSEQUENCE
1	Profile exists	Missing profile is `NOT_FOUND`.
2	Eligibility	Revoked: “Your interview eligibility has been revoked by interview operations…” Not eligible: complete outstanding skills.
3	No active booking	Only one CONFIRMED booking per student at a time.
4	Badge state	AWARDED: “You already hold the badge.” SUSPENDED: new bookings are paused.
5	Attempts	Attempts used ≥ interview type maximum → operations contact.
6	Waiting period	After a FAIL / RETAKE_LATER, booking is blocked until `retakeEligibleFrom` unless operations approved an early retake.
7	Slot rules	Slot must exist, be OPEN, start in the future and give at least 60 minutes of lead time.

The pure rule function is called once for display and again inside the booking transaction after the student and slot rows are locked, so a stale screen cannot bypass a rule.

SECTION 06

06 Booking under contention

This is the correctness centrepiece of the assignment: two students race for one slot and exactly one wins. The implementation uses three independent defences.

1. One transaction with row locks

```
// src/lib/booking.ts — inside prisma.$transaction
SELECT id FROM "StudentProfile" WHERE id = $studentId FOR UPDATE;

// re-run eligibility after taking the lock
const check = await getBookingEligibility(tx, studentId, typeId, now);

SELECT id FROM "InterviewSlot" WHERE id = $slotId FOR UPDATE;
// verify status = 'OPEN', startsAt > now, lead time >= 60 min

UPDATE "InterviewSlot" SET status = 'BOOKED'
 WHERE id = $slotId AND status = 'OPEN';   // must affect exactly 1 row
```

The student lock serialises a student's own requests; the slot lock serialises everyone competing for that slot. Eligibility and slot state are re-read under the lock, so neither a cached page nor a simultaneous cancellation can produce an invalid booking. If the conditional update does not affect exactly one row, the transaction aborts with a conflict.

2. Partial unique indexes

```
CREATE UNIQUE INDEX one_active_booking_per_student
  ON "InterviewBooking" ("studentId") WHERE status = 'CONFIRMED';

CREATE UNIQUE INDEX one_active_booking_per_slot
  ON "InterviewBooking" ("slotId") WHERE status = 'CONFIRMED';
```

These are a database-level backstop. Even if a future code path forgets a lock, the invariant holds: at most one confirmed booking per student and per slot. Historic rows (cancelled, rescheduled, no-show, completed) do not participate, which is why the indexes are partial rather than global.

3. Idempotency

The client generates a key per booking attempt. `InterviewBooking.idempotencyKey` is unique. A retried request returns the existing booking and interview with `replayed: true` instead of creating a second booking. A key reused by a different student is rejected.

Failure mapping

CONDITION	CODE	MESSAGE BEHAVIOUR
Slot no longer OPEN	`CONFLICT`	"That slot was just taken by another student…" — availability refreshes
Unique index on slot	`CONFLICT`	Same race message
Unique index on student	`CONFLICT`	"You already have a confirmed interview booking."
Lead time under 60 minutes	`POLICY`	Names the preparation requirement
Eligibility revoked	`FORBIDDEN`	Operations reason and coordinator contact
Wait period / attempts	`POLICY`	Shows the eligible date or attempt limit

CONDITION	CODE	MESSAGE BEHAVIOUR
Not found	`NOT_FOUND`	Slot or profile missing

Cancellation, rescheduling and no-shows

OPERATION	RULES
Cancel	Student may cancel only their own booking, only before the start, and at least 4 hours ahead. Operations may cancel at any time. Blocked once an evaluation or result exists. Booking becomes `CANCELLED` with reason and timestamp; a future slot returns to `OPEN`; the interview row is removed; the student is notified; the action is audited.
Reschedule	Only a CONFIRMED, future, unevaluated booking. Student needs the 4-hour window; operations does not. Both slot rows are locked in sorted id order to avoid deadlocks. The new slot is reserved first, then the old one released. The old booking becomes `RESCHEDULED` and the new booking links to it via `rescheduledFromId`. A conflict leaves the original booking untouched.
No-show	Operations only. Confirmed booking → `NO_SHOW`; interview → `COMPLETED`; student notified; slot is not released; audited.

ONE DELIBERATE CONSEQUENCE

Cancelling or rescheduling deletes the interview record attached to the old booking, because the interview depends on that booking. The booking row, its status, reason, timestamp, reschedule link and audit entries preserve the history. Published results are never deleted; those interviews cannot be moved.

SECTION 07

07 Availability and slot generation

Window rules

- Date must be today or later in the application timezone; times use `HH:MM`; end must be after start.
- Slot length is 15–180 minutes (UI and seed default to 45).
- A break needs both endpoints, must sit inside the window, and end after it starts.
- Windows for the same interviewer and date may not overlap. Duplicate `(interviewer, date, startTime)` is also rejected by a unique index.
- Editing a window that contains a confirmed booking is refused: operations must reassign those interviews first.
- Created slots that have already passed are not generated; a window with no future slots is rejected.

Generation

`buildSlotRange` walks the window in fixed steps and skips any slot that intersects the break. Each boundary is converted from IST wall-clock time to a UTC instant, so slots remain correct across a server in another timezone and across daylight-saving changes in the application zone (IST does not observe DST). Each slot stores `startsAt`, `endsAt`, the interviewer and the source window. A unique index on `(interviewerId, startsAt)` prevents duplicate slots even if two windows are created concurrently.

Blocking and reopening

ACTION	EFFECT
Block window	Window status becomes `BLOCKED`; future OPEN slots become `CANCELLED`; confirmed bookings are not touched. Those interviews enter the operations reassignment queue while their availability is blocked. The audit summary states how many interviews need reassignment.
Unblock	Window returns to `ACTIVE`; future cancelled slots with no confirmed booking are reopened and missing slots are recreated. Audit records how many were restored.

ACTION	EFFECT
Delete	Window is removed only when it has no confirmed bookings; audit records the deletion.

Interviewer capacity (`maxDailyInterviews`, default 6) is displayed in workload views but is not enforced as a hard booking rule. This is documented rather than implied.

SECTION 08

08 Evaluation and publication

Evaluation submission

- Only the assigned interviewer may evaluate. Ownership is checked inside the service after the interview row is locked, not just in the page.
- Three integer scores, 0–100 each. Overall score is the rounded mean.
- Strengths and improvements require at least 10 characters; internal notes are optional up to 2000; recommendation is PASS, FAIL or RETAKE_LATER.
- One evaluation per interview. A second submission is a `CONFLICT`.
- On success: interview → `RESULT_PENDING`, booking → `COMPLETED`, a scheduled retake becomes `COMPLETED`, the student is notified that the result awaits publication, and the scores plus recommendation are audited.
- The UI warns when the session is still in the future. This is a warning, not a hard clock block; the domain permits evaluation at any legal interview state for operational flexibility.

Publication

- Operations must supply a feedback summary of at least 10 characters.
- The interview is locked; an evaluation is required; a result may exist only once; the transition to `RESULT_PUBLISHED` must be legal.
- A PASS result upserts a badge award as `AWARDED` with an explanatory note and sends a badge notification.
- A FAIL or RETAKE_LATER result sets `retakeEligibleFrom = publishedAt + interviewType.retakeWaitDays`.
- The interview stores the outcome, score and publication time. The student receives the exact assignment copy for that outcome.
- Publication, badge award, notification and audit all commit together. If any step fails, none are written.

INTERVIEW TYPE	DURATION	PASSING SCORE	RETAKE WAIT	MAX ATTEMPTS
GRIT Technical Interview	45 min	70	14 days	3
GRIT Systems Deep Dive	45 min	75	21 days	2

The passing score is stored on the interview type for policy completeness. The interviewer's recommendation is the authoritative outcome in this prototype; the UI does not auto-derive it from scores. That keeps human judgement explicit and auditable.

SECTION 09

09 Badges, retakes and integrity

Badge lifecycle

Badges are awarded by publication, not by the interviewer. Operations can later suspend or nullify a badge with a mandatory note; restoring a suspended badge is also audited. Every badge change sends an in-app notification and leaves the interview result untouched, which matters contractually: a revoked badge must not erase the evidence of what happened.

Retakes

1. A student may request a retake only after a published FAIL or RETAKE_LATER, with a reason of at least 15 characters.
2. Only one PENDING request per interview type at a time.
3. The request stores the result-derived `eligibleFrom` date.
4. Operations approves or rejects with a note of at least 5 characters. Approval permits booking before the waiting period ends; rejection leaves the original date.
5. When the retake is booked, the approved request becomes SCHEDULED; when the new interview is evaluated, it becomes COMPLETED.
6. Attempts are counted from interviews, so a retake consumes an attempt against the type maximum.

Malpractice

STEP	RULES
Report	Operations records severity (LOW/MEDIUM/HIGH), an allegation of at least 20 characters and optional text evidence. Case starts REPORTED.
Review	Case moves to UNDER_REVIEW. Notifications and audit follow each status.
Confirm	Requires a resolution and a badge action (NONE, SUSPEND, NULLIFY). If an action is chosen, the matching badge award must exist; the badge service applies the transition with a case-referencing note.
Reject / close	Rejection notifies the student that no adverse finding was made. Closed cases are terminal.

The original result, evaluation and publication record are preserved in every case. Integrity action changes the badge state, not the truth of the interview record.

Eligibility administration

Operations can revoke eligibility with a reason, and restore it later. Revocation is sticky: completing skills does not silently restore a revoked student. Both actions are audited and notified. Automatic eligibility is derived from required skills by the same pure calculation used on the dashboard.

SECTION 10

10 Notifications and exact copy

Notifications are first-class rows, not email side effects. They carry a type, title, body, deep link, read timestamp and creation time. The bell shows unread state; the notifications page supports mark-one and mark-all. Each notification is written inside the same transaction as the change it describes.

Notification types

- `ELIGIBILITY_UNLOCKED`
- `BOOKING_CONFIRMED`
- `BOOKING_RESCHEDULED`
- `BOOKING_CANCELLED`
- `INTERVIEW_REMINDER`
- `INTERVIEW_COMPLETED`
- `RESULT_PUBLISHED`
- `BADGE_AWARDED`
- `RETAKE_UPDATED`
- `MALPRACTICE_UPDATE`
- `SYSTEM`

Reminder types exist in the schema but no scheduler sends them; the product deliberately does not claim email or SMS delivery.

Required student messages, exactly as written

These constants live in `src/lib/notifications.ts` and are asserted verbatim by the test suite.

> **1 · ELIGIBILITY UNLOCKED**
> "Congratulations! You have completed the required skill milestones and are now eligible for your GRIT interview. Choose an available slot to begin your badge clearance journey."

> **2 · BOOKING CONFIRMED**
> "Your GRIT interview has been successfully booked. Your interview details are available in your dashboard. Please review your scheduled date and time and be ready before the interview begins."

> **3 · PASSED**
> "Congratulations! You have successfully cleared your GRIT interview. Your badge has been awarded and is now available in your dashboard."

> **FAIL VARIANT**
> "Your GRIT interview result has been published. You have not cleared this attempt. Please review your feedback and follow the retake instructions shown in your dashboard."

> **RETAKE-LATER VARIANT**
> "Your interview has been evaluated. A further attempt is recommended after the specified waiting period. Please check your dashboard for the next eligible date and instructions."

Other transactional copy is written in the same voice: booking cancellation explains whether the slot was released, rescheduling names the new time and interviewer and confirms the old booking is preserved, and a no-show or integrity outcome always states what remains on record.

SECTION 11

11 Audit and history

`AuditLog` is append-only and written inside the transaction that performs the change. Each row records the actor (nullable for system events), actor role, action key, entity type and id, a human summary, and JSON before/after values.

ACTION KEY	WRITTEN WHEN
`booking.created / cancelled / rescheduled / reassigned / no_show`	Every booking mutation, including operations reassignment

ACTION KEY	WRITTEN WHEN
`interview.started`	Interviewer opens the session
`result.submitted`	Evaluation with scores and recommendation
`result.published`	Outcome, score, retake date
`badge.status_changed`	Every award, suspension, nullification or restore
`retake.requested / approved / rejected`	Student request and operations decision
`malpractice.reported / under_review / confirmed / rejected / closed`	Each integrity transition
`eligibility.unlocked / recalculated / revoked / restored`	Derived and administrative eligibility changes
`availability.created / updated / blocked / unblocked / deleted`	Schedule changes, including reassignment counts

How history is preserved without mutable ledgers

- Rescheduling keeps the old booking as `RESCHEDULED` and links the replacement through `rescheduledFromId`.
- Cancelling keeps the booking row with reason and timestamp; the slot is either reopened or cancelled depending on whether its time has passed.
- Interview results are terminal. Badge state moves; the result does not.
- Integrity cases retain allegation, evidence, resolution, severity and deciding user.
- Audit rows survive user deletion through `onDelete: SetNull` on the actor, so the record outlives the account.

SECTION 12

12 Data model reference

PostgreSQL through Prisma 7 and `@prisma/adapter-pg`. The generated client lives in `src/generated/prisma`. Migrations are committed; the booking-integrity migration adds the partial indexes and check constraints.

MODEL	KEY FIELDS	CONSTRAINTS AND INDEXES
User	email, name, role, timezone	unique email; index role
StudentProfile	userId, studentCode, program, cohort, university, eligibilityStatus, eligibilityReason, eligibleAt, revokedAt	unique userId, studentCode; index eligibilityStatus
InterviewerProfile	userId, title, specialty, bio, timezone, maxDailyInterviews, isActive	unique userId; index isActive
Skill	code, name, description, sortOrder, isRequired	unique code
StudentSkillProgress	studentId, skillId, status, score, completedAt	unique (student, skill); index (student, status)
InterviewType	code, name, durationMinutes, passingScore, retakeWaitDays, maxAttempts, isActive	unique code
Badge	code, name, level, validityMonths, interviewTypeId	unique code; index type
InterviewerAvailability	interviewerId, date, startTime, endTime, slotMinutes, breakStart, breakEnd, status, note	unique (interviewer, date, startTime); index (interviewer, date)
InterviewSlot	availabilityId, interviewerId, startsAt, endsAt, status	unique (interviewer, startsAt); index (status, startsAt)
InterviewBooking	studentId, slotId, status, attemptNumber, idempotencyKey, bookedAt, cancelledAt, cancellationReason, rescheduledFromId	unique idempotencyKey; indexes (student,status), (status,bookedAt); partial uniques per student/slot
Interview	bookingId, studentId, interviewerId, interviewTypeId, status, outcome, attemptNumber, retakeEligibleFrom, startedAt, completedAt, resultPublishedAt	unique bookingId; indexes on interviewer/status, student/status, status/resultPublishedAt
InterviewEvaluation	technicalScore, problemSolvingScore, communicationScore, overallScore, strengths, improvements, notes, recommendation, submittedByUserId, submittedAt	unique interviewId; index submittedAt
InterviewResult	outcome, overallScore, feedbackSummary, publishedAt, publishedByUserId	unique interviewId; index publishedAt
BadgeAward	studentId, badgeId, interviewId, status, awardedAt, statusNote	unique (student, badge), unique interviewId; index status
RetakeRequest	studentId, interviewId, interviewTypeId, reason, status, eligibleFrom, reviewedByUserId, reviewedAt, decisionNote	index (status, createdAt), studentId
MalpracticeCase	studentId, interviewId, reportedByUserId, status, severity, allegation, evidence, resolution, badgeAction, confirmedByUserId, confirmedAt	index (status, createdAt), studentId
Notification	userId, type, title, body, link, readAt, createdAt	indexes (userId, readAt), (userId, createdAt)
AuditLog	actorUserId, actorRole, action, entityType, entityId, summary, previousValue, newValue, createdAt	indexes createdAt, (entityType, entityId), actorUserId

Referential choices

- A booking's slot uses `onDelete: Restrict`: a slot cannot disappear under a booking.
- An interview requires a booking (`Restrict`) and a type (`Restrict`); evaluations, results and awards cascade from their interview.
- Users cascade to profiles but audit actors are set to null on deletion, preserving the trail.
- Actionable money-like history is never mutated: status transitions create new rows or use terminal statuses.

SECTION 13

13 Server actions catalogue

Every mutation is a server action in `src/app/actions/*`. Each authenticates, validates with Zod, calls a domain service, returns a typed `ActionResult`, and revalidates affected routes. The client shows success or the returned domain error as a toast.

ACTION	ROLE	VALIDATION AND EFFECT
`bookSlotAction`	Student	Non-empty slot; idempotency key; atomic booking
`cancelBookingAction`	Student	Booking id; optional reason ≤500; policy window enforced in service
`rescheduleBookingAction`	Student	Booking and new slot required; 4-hour window in service
`requestRetakeAction`	Student	Type id; reason ≥15
`createAvailabilityAction`	Interviewer	Date, HH:MM times, 15–180 minutes; overlap, break and past-date rules
`updateAvailabilityAction`	Interviewer	Same validation; refuses windows with confirmed bookings
`toggleAvailabilityAction`	Interviewer	Block/unblock; cancels or restores future open slots
`deleteAvailabilityAction`	Interviewer	Ownership and no-booking guard
`startInterviewAction`	Interviewer	Assigned interviewer only; state transition
`submitEvaluationAction`	Interviewer	Scores 0–100 integers; strengths/improvements ≥10; valid recommendation; one evaluation
`publishResultAction`	Operations	Feedback ≥10; evaluation required; single publication
`reassignInterviewAction`	Operations	Booking and new slot; reschedule service bypasses the student window
`markNoShowAction`	Operations	Booking required; confirmed booking only
`adminCancelBookingAction`	Operations	Reason ≥5; no policy window
`revokeEligibilityAction`	Operations	Reason ≥10; audited
`restoreEligibilityAction`	Operations	Reason ≥10; audited
`reviewRetakeAction`	Operations	APPROVED/REJECTED; note ≥5
`reportMalpracticeAction`	Operations	Severity enum; allegation ≥20; evidence ≤2000
`updateMalpracticeAction`	Operations	Status enum; resolution ≤2000; badge action
`setBadgeStatusAction`	Operations	AWARDED/SUSPENDED/NULLIFIED; note ≥5; state machine
`markNotificationReadAction`	Any session	Own notification only
`markAllNotificationsReadAction`	Any session	Own unread rows only
`demoLogin`	Public (demo)	Known persona email; issues signed cookie; disabled outside demo mode
`logout`	Any session	Deletes cookie
`resetDemoAction`	Operations (demo)	Requires ADMIN and DEMO_MODE; truncates and reseeds; reissues admin session

> **ERROR CONTRACT**
> `DomainError` codes are VALIDATION, FORBIDDEN, NOT_FOUND, CONFLICT, POLICY and STATE. Unknown failures are logged server-side and returned as a generic retry message, so internal details never leak to the browser.

SECTION 14

14 Analytics and metrics

All figures are computed from PostgreSQL at request time over a configurable range (default 30 days). Nothing on the analytics screen is hard-coded.

Operations KPIs

METRIC	DEFINITION
Total / eligible students	All student profiles; profiles whose eligibility is ELIGIBLE
Eligible but unbooked	Eligible profiles with no CONFIRMED booking — the primary operations queue
Confirmed bookings	Bookings currently CONFIRMED
Completed interviews	Interviews completed, pending result or published with completion inside the range
Pass rate	Published PASS ÷ all published results (null when empty)
Retake rate	Published FAIL or RETAKE_LATER ÷ all published results
No-show rate	No-shows ÷ (completed bookings + no-shows) over the range
Average time to interview	Mean days from eligibility to slot start
Average publication lag	Mean hours from interview completion to result publication
Interviewer utilisation	Booked upcoming slots ÷ all upcoming slots
Eligibility conversion	Students who have ever booked ÷ eligible students
Overdue results	RESULT_PENDING longer than 24 hours since completion
Pending retakes / open integrity cases	Current open queues
Previous-week utilisation	Booked slots ÷ all slots that started in the last 7 days

Charts and queues

- **Bookings trend:** one point per day for the selected range, counted by booking creation time in the application timezone.
- **Outcomes:** Pass (teal), Fail (rose), Retake later (indigo), from published results.
- **Workload:** per interviewer — upcoming, in progress, pending publication, capacity and utilisation.
- **Queues:** eligible-unbooked with days waiting, reassignments (inactive interviewer or blocked availability), retake requests, integrity cases, upcoming interviews, recent audit, repeated cancellers in the last 30 days.
- **Student dashboard:** skills, eligibility, active interview, upcoming and historical interviews, latest published result, badge, retakes and recent notifications.
- **Interviewer dashboard:** today's sessions, next eight upcoming, interviews needing evaluation, awaiting publication, completed history, slots, open/booked counts and utilisation.

SECTION 15

15 Security and access control

LAYER	IMPLEMENTATION
Session	HMAC-SHA256 signed payload `{sub, exp}`; 7-day expiry; timing-safe signature comparison. Cookie is httpOnly, SameSite=Lax, path=/, and Secure in production.
Identity	Demo persona picker resolves a seeded user and issues the cookie. `DEMO_MODE=false` disables login and reset. No passwords are stored.
Route guards	Every authenticated page calls `requireUser`, `requireRole`, `requireStudent` or `requireInterviewer`. Wrong role redirects to that role's home; no session redirects to login.
Action guards	Every server action repeats the guard. A client cannot invoke an admin action with a student cookie.
Ownership	Students can only cancel/reschedule their own bookings; interviewers can only open and evaluate interviews assigned to them. Checked inside the transactional services.
Input validation	Zod schemas at the action boundary plus domain validation (scores, lengths, times, enums, state transitions).
Output safety	React escaping by default; no `dangerouslySetInnerHTML`.
SQL safety	Prisma parameterises queries; raw statements use tagged template parameters.
Error hygiene	Unknown exceptions are logged server-side and replaced with a generic public message.
Demo reset	Requires an authenticated ADMIN and demo mode; performs a scoped TRUNCATE of application tables only.
Public data	`/result-impact` is the only page that reads student rows without a session. It is read-only, demo-only and 404s outside demo mode.

HONEST BOUNDARY

This is not production identity: there is no login rate limiting, SSO, password policy or multi-device session revocation. The demo-mode switch is the isolation boundary, and replacing the picker with an identity provider is the first production task.

SECTION 16

16 Time handling

GRIT schedules in India Standard Time. Slots are stored as UTC instants, created from IST wall-clock windows and displayed in the configured application timezone.

- `APP_TIMEZONE` (default `Asia/Kolkata`) is the single scheduling authority.
- `zonedDateTimeToUtc` converts a date key and HH:MM to an instant using the zone offset for that date.
- `dateKeyInTz` groups bookings and "today" correctly regardless of server timezone.
- `formatDateTime`, `formatLongDate` and `tzAbbreviation` own all user-facing dates.
- Lead time and cancellation windows compare instants (`hoursUntil`), never formatted strings.
- Users carry a `timezone` field; the assignment scope keeps creation in IST and display consistent with the configured zone.

Adding multi-timezone availability is a domain change: windows would need their own zone, slot generation would use it, and "today" would become per interviewer. The UI would not need to change.

SECTION 17

17 Testing

44 tests across 11 suites run with Node's test runner and `tsx`. Integration tests use a dedicated PostgreSQL database (`grit_test`) migrated before the run. Unit tests cover pure rules and time maths.

SUITE	TESTS	WHAT IT PROVES
booking.integration	9	Exactly one winner in a two-student race; idempotent replay; one booking per student; 60-minute lead; revoked eligibility; cancellation window vs operations; ownership; reschedule history; no-show without release
lifecycle.integration	8	Pass publication, badge award and exact notification copy; foreign/second evaluation; invalid scores; publish before evaluation; double publication; wait period then approved early retake; malpractice nullification preserving the result; eligibility revoke/restore with audit
notifications	5	All five student messages match the assignment verbatim
session-token	5	Round trip, wrong secret, tampered payload, expiry, malformed input
state-machine	10	Every documented move, terminal states, illegal transitions, humanised labels
time	7	IST to UTC conversion, day boundary, formatting, contiguous slots, break skipping, duration maths

Verification beyond tests

- `npm run typecheck`, `npm run lint`, `npm test` and `npm run build` all pass on the committed tree.
- A headless Chrome harness logs in as each role, exercises booking, confirms the dialog writes nothing, checks the result page at 1440px and 390px, and fails on console errors or horizontal overflow.
- The server is run from the production build for these checks, not only from the dev server.

SECTION 18

18 Interface and accessibility

Visual system

The product reads as quiet enterprise software: a warm canvas, ink text, dense but legible tables, and status pills that use the domain vocabulary. Each role owns an accent — student teal, interviewer indigo, operations rose — applied through CSS custom properties on a single shell, so the identity changes without forking components. Gradients are restrained: hairline card borders, role-tinted icons, a drifting ambient wash on the landing page and key dashboards. Motion is limited to 150–300 ms entrance and progress effects.

Decision-point design

- **One next step** on the student home, with an explanation and the resolving link.
- **Preview mode** for blocked students: availability is visible, booking is not implied.
- **Confirmation dialog** with slot, interviewer and cancellation policy; the slot is reserved only on confirm.
- **Specific errors** from the domain, surfaced as toasts, with refreshed availability after a lost race.
- **Navigation by task**: role home, main task, history, notifications; a command menu opens with Cmd/Ctrl+K.

Accessibility

- Skip link to main content; visible focus rings; semantic landmarks and headings.

- Native modal behaviour for confirmations and the mobile drawer, labelled dialogs, Escape to close, focus containment and scroll lock.
- Touch targets in the 44px class; mobile bottom navigation with a “More” drawer; layouts verified down to 390px.
- Status is never communicated by colour alone: pills carry text, and journey stages use labels plus a “You are here” marker.
- `prefers-reduced-motion` removes entrance, drift and pulse animations.

SECTION 19

19 Operating runbook

Environment

VARIABLE	REQUIRED	PURPOSE
DATABASE_URL	Yes	PostgreSQL connection string (pooled at runtime on serverless)
AUTH_SECRET	Yes	HMAC key for session tokens; generate with `openssl rand -hex 32`
APP_TIMEZONE	No	Scheduling zone; defaults to `Asia/Kolkata`
DEMO_MODE	No	Defaults to `true`; enables persona login, reset and result comparison
NEXT_PUBLIC_APP_NAME	No	Display name used by the shell

Commands

```
npm install                 # postinstall runs prisma generate
npm run db:migrate          # development migrations
npm run db:seed             # seed the 23 persona scenarios
npm run dev                 # local development
npm run typecheck && npm run lint && npm test && npm run build
npm start                   # serve the production build
npm run db:reset-demo       # truncate and reseed demo data (also available to admins)
npm run db:deploy           # apply committed migrations in a hosted environment
```

Reset behaviour

The reset action is available to administrators only in demo mode. It truncates the 18 application tables with `RESTART IDENTITY CASCADE` and reseeds in one sequence, then reissues the admin session so the operator is not logged out. It is intentionally not a general-purpose migration tool.

Deployment path

1. Create a hosted PostgreSQL database (Supabase or Neon). Keep the direct URL for migrations and the pooled URL for runtime.
2. Run `DATABASE_URL="…direct…" npm run db:deploy` then `npm run db:seed` once.
3. Create a Vercel project from the repository and set the variables above; `DATABASE_URL` uses the pooled string (Supabase transaction pooler on port 6543 with `?pgbouncer=true&connection_limit=1`).
4. Deploy with `vercel --prod`, then verify all three roles, a booking attempt and the result page on the public URL before sharing it.
5. Migrations are deliberately not part of the build: a deploy cannot silently reshape the database.

> **CURRENT DEPLOYMENT STATE**
>
> Hosting is prepared but not published in this environment; the Vercel CLI is unauthenticated and no hosted database credentials are present. The runbook above is the exact sequence used once those credentials exist. No URL is claimed.

SECTION 20

20 Limitations and next steps

LIMITATION	WHY IT EXISTS	PRODUCTION STEP
Demo authentication	Assignment scope; no identity provider available	Replace the picker with SSO/OIDC; keep session signing and guards
Single-timezone scheduling	GRIT operates in IST; the brief is about scheduling correctness, not global calendars	Zone per availability window, per-interviewer “today”, zone-aware conflict checks
In-app notifications only	No external provider configured	Email/SMS worker behind the notification service; retries and delivery status
No file uploads	Evidence is captured as operations-entered text	Object storage with signed URLs, malware scanning, retention policy
Capacity not enforced	`maxDailyInterviews` is informational in the demo	Reject bookings that exceed the interviewer’s daily cap; surface in availability
No background jobs	Reminder types exist but nothing schedules them	Queue for reminders, overdue publication escalation and stale holds
Evaluation timing is a warning	Operations may need to record an evaluation late or early	Configurable clock policy per interview type

What I would do next, in order

1. Production identity and audit export (the trust boundary).
2. Background delivery and escalation (the operational gap).
3. Multi-timezone availability (the scale gap).
4. Capacity enforcement and slot hold expiry (the contention refinement).

SECTION 21

21 Interview questions and answers

Walk me through two students clicking Book at the same time.

Both transactions lock their own student profile and re-check eligibility. Both try to lock the same slot row; one waits. The winner sees OPEN, reserves it with a conditional update that must affect exactly one row, creates the booking and interview, and commits. The loser re-reads BOOKED and gets a conflict with the “slot was just taken” message. If the lock were ever bypassed, the partial unique index on slotId rejects the second confirmed booking.

Why use both row locks and unique indexes?

Locks provide a clean, specific failure and make eligibility checks correct under contention. Indexes guarantee the invariant regardless of future code paths. The rule that two people cannot hold one seat is important enough to defend twice.

Why server actions instead of a REST API?

One deployment and no serialization layer to drift out of sync. Zod validation, RBAC and the mutation run in one place. The services underneath are API-shaped, so extracting HTTP endpoints later is mechanical.

How is history preserved when a booking changes?

Cancellation keeps the booking row with reason and timestamp. Rescheduling marks the old booking RESCHEDULED and links the new one through rescheduledFromId. Results are terminal; badge changes move status but never delete the result. Every step writes an audit row with before and after values in the same transaction.

What happens if an interviewer submits twice or publishes twice?

Both operations lock the interview and refuse a second write: one evaluation per interview, one result per interview. The tests assert the conflict.

How do you know the student messages are exact?

They are constants, not inline strings. The notification service consumes them, the write-up quotes them, and five tests compare them verbatim.

What happens when an interviewer blocks a day that has bookings?

Future open slots are cancelled, confirmed bookings are untouched, and those interviews appear in the operations reassignment queue because their availability is blocked. Operations reassigns them to another open slot using the same reschedule service, bypassing the student’s four-hour window.

Why can operations publish but interviewers cannot?

Separation of duties. The interviewer records evidence and a recommendation; operations owns the contractual publication, badge award and retake clock. It also gives the overdue-publication queue a real meaning.

How does the retake waiting period work?

Publishing a non-pass sets retakeEligibleFrom to publication plus the type’s wait. Booking checks that date and blocks with the exact eligible date. A student can request an early review; if operations approves, the booking check accepts before the date, and the request moves to SCHEDULED when booked.

What did you deliberately leave out?

Production identity, external messaging, multi-timezone creation and file uploads. Each is isolated behind a service boundary rather than faked. I would fix identity first because it is the largest realism gap.

How was this verified?

Typecheck, lint, 44 tests including a real two-writer database race, a production build, and a headless browser pass across all three roles at desktop and 390px that fails on console errors or overflow. I do not treat a green build as proof of feature correctness.

APPENDIX A

A Policy constants

Minimum booking lead	60 minutes before the slot starts
Cancellation / reschedule window	4 hours before the slot starts (students); operations unrestricted
Default interview length	45 minutes (allowed 15–180)
Round 1 retake wait	14 days
Round 1 maximum attempts	3
Advanced type retake wait	21 days
Advanced type maximum attempts	2
Badge validity	24 months (stored on the badge)
Evaluation scores	Integer 0–100 on three dimensions; overall is the rounded mean
Minimum evidence	Strengths/improvements 10 characters; retake reason 15; decision notes 5; allegation 20; feedback summary 10
Session lifetime	7 days, HMAC-signed, httpOnly
Overdue publication	More than 24 hours after interview completion
Analytics range	30 days by default
Demo identities	18 students, 4 interviewers, 1 operations administrator

APPENDIX B

B Source map

PATH	RESPONSIBILITY
`prisma/schema.prisma`	All models, enums and indexes
`prisma/migrations/…_booking_integrity`	Partial unique indexes and checks
`src/lib/demo/seed.ts`	Types, badges, skills and 23 personas
`src/lib/demo/reset.ts`	Scoped truncate and reseed
`src/lib/state-machine.ts`	Transition tables and assertions
`src/lib/booking.ts`	Atomic book, cancel, reschedule, no-show
`src/lib/booking-eligibility.ts`	Pure eligibility rules, policy constants
`src/lib/availability.ts`	Windows, overlap, generation, blocking
`src/lib/interviews.ts`	Start and evaluation
`src/lib/results.ts`	Publication, badge award, result copy
`src/lib/retakes.ts`	Request and review
`src/lib/malpractice.ts`	Integrity workflow and badge action
`src/lib/badges.ts`	Badge state service
`src/lib/eligibility.ts / eligibility-admin.ts`	Derived eligibility and administrative revoke/restore
`src/lib/notifications.ts`	Exact copy and transactional notify helper
`src/lib/audit.ts`	Append-only audit writer
`src/lib/auth.ts / session-token.ts`	Cookie session, HMAC, RBAC guards
`src/lib/analytics.ts`	KPIs, charts, queues, dashboards
`src/lib/student-journey.ts`	Shared six-stage path and next action
`src/lib/time.ts`	IST conversion, formatting, slot range
`src/app/actions/*`	Server actions by role
`src/app/(app)/…`	Authenticated role screens
`src/app/result-impact`	Before/after result comparison
`src/components/*`	Shell, UI primitives, charts, journey, dialogs
`tests/*`	Unit and PostgreSQL integration suites
`docs/WRITE-UP.md`	Two-page product and technical write-up

> **DOCUMENT HONESTY**
> Every rule, constant, index and route in this manual was read from the committed implementation at the time of writing. Where the product does not do something — production identity, external messaging, multi-timezone creation, hosted deployment — the document says so explicitly.